

COMPLETE STUDY GUIDE & PRACTICE TEST

Anthropic Claude Certification Prep

Master Claude models, API, prompt engineering, tool use, safety, and pricing. Includes code examples, diagrams, and 50 exam-style MCQs with explanations.

Topics: 11 sections MCQs: 50 questions Source: Official Anthropic Docs Updated: September 2026

Important Note

As of September 2026, Anthropic has not publicly announced a certification called "CCAO-F." Anthropic offers free courses via **Claude Academy** (academy.claude.com). This guide covers all foundational knowledge from official documentation, suitable for any Claude assessment.

TABLE OF CONTENTS

01 Anthropic & Claude Overview	02 Claude Model Family
03 Platforms & Deployment	04 Pricing Deep Dive
05 Messages API Fundamentals	06 Tool Use & Function Calling

07 Vision & Multimodal

08 Thinking & Reasoning

09 Prompt Engineering

10 Safety, Ethics & Data Policies

11 Practice Test — 50 MCQs

PART 01

Anthropic & Claude Overview

What is Anthropic?

Anthropic is an AI safety company founded in 2021 that builds AI systems designed to be safe, beneficial, and understandable. Their flagship product is the **Claude** family of large language models.

Anthropic's Core Belief

"AI's impact could rival the industrial revolution, potentially arriving within a decade, but we aren't confident it will go well."

What is Claude?

Claude is a family of state-of-the-art LLMs that excels at language, reasoning, analysis, coding, multilingual tasks, vision, and tool use. Anthropic offers two primary building paths:

Path	What It Is	Best For
Messages API	Direct model prompting access	Custom agent loops, fine-grained control
Claude Managed Agents	Pre-built configurable agent harness	Long-running tasks, async work

Key Products & Platforms

Product	Description
claude.ai	Consumer chat interface for interacting with Claude
Claude API	Developer API for building applications (platform.claude.com)
Claude Console	Browser-based playground for testing prompts and API calls
Claude Code	Official CLI for coding assistance in the terminal
Claude Academy	Free courses at academy.claude.com
Claude Agent SDK	SDK for building custom agents

Study Resources

Official Documentation

platform.claude.com/docs — Primary source for API, models, features, and guides.

Claude Academy

academy.claude.com — Free courses: "AI Fluency," AI capabilities/limitations, human-agent teams.

Prompt Engineering Tutorial

github.com/anthropics/prompt-eng-interactive-tutorial — Interactive, example-filled tutorial.

Claude Cookbook

platform.claude.com/cookbook — Jupyter notebooks for PDFs, embeddings, tool use, and more.

PART 02

Claude Model Family — Deep Dive

Current Model Lineup (September 2026)

Model	API ID	Best For	Context	Max Out	Input	Output	Speed
Fable 5.1	claude-fable-5-1	Demanding reasoning, long-horizon agentic	1M	128K	\$10	\$50	Slower
Opus 5.5	claude-opus-5-5	Agentic coding & knowledge work	1M	128K	\$4	\$20	Moderate
Sonnet 5	claude-sonnet-5-5	Speed + intelligence balance	1M	128K	\$2	\$10	Fast
Haiku 4.5	claude-haiku-4-5-20251001	Fastest, lowest cost, high volume	200K	64K	\$1	\$5	Fastest

All prices in USD per million tokens (MTok). All models support text + image input, text output, multilingual, vision, and tool use.

Model Selection Decision Tree

Most capable reasoning / multi-day agentic tasks?

→ Fable 5.1

Complex coding & long-running agent sessions?

→ Opus 5.5

Everyday tasks, best speed-to-intelligence ratio?

→ Sonnet 5

High volume, fastest speed, lowest cost?

→ Haiku 4.5

Unsure where to start?

→ Opus 5.5 (recommended default)

Key Facts to Remember

- **1M tokens** $\approx$ 555,000 words (new tokenizer from Claude 4.7+)
- **Haiku 4.5's 200K** $\approx$ 150,000 words
- **Tokenizer change:** Claude 4.7+ produces $\sim$ 30% more tokens for the same text
- **Knowledge cutoffs:** Fable 5.1 & Opus 5.5 $\rightarrow$ June 2026; Sonnet 5 $\rightarrow$ Jan 2026; Haiku 4.5 $\rightarrow$ Feb 2025
- **Effort parameter:** Fable 5.1 defaults to "high," Opus 5.5 to "medium," Sonnet 5 to "high." Not supported on Haiku 4.5.

Exam Tip

The model names follow a pattern: **Fable** = most capable, **Opus** = complex/agentive, **Sonnet** = balanced, **Haiku** = fastest/cheapest. Numbers indicate generation.

PART 03

Platforms & Deployment

Five Platforms for Claude

Platform	Operator	Data Processor
Claude API	Anthropic (first-party)	Anthropic
Amazon Bedrock	AWS	AWS
Claude Platform on AWS	Anthropic (on AWS infra)	Anthropic
Google Cloud (Vertex AI)	Google	Google
Microsoft Foundry	Anthropic (on Azure)	Anthropic

Feature Availability Labels

Label	Meaning
Stable	Fully supported, production-ready, standard API versioning
Beta	Preview for feedback, may change or be discontinued
Deprecated	Still functional, migration path provided
Retired	No longer available

Claude Consumption Units (CCUs)

Used on Claude Platform on AWS and Microsoft Foundry:

- 1 CCU = \$0.01 USD (fixed)
- Token usage rated at standard rates, then converted to CCUs
- Billed hourly via marketplace, monthly invoices
- Postpaid (arrears) only — no prepaid credits

PART 04

Pricing — Complete Reference

Prompt Caching

Operation	Cost Multiplier	Duration	Break-Even
5-min cache write	1.25x base input	5 minutes	After 1 read
1-hour cache write	2x base input	1 hour	After 2 reads
Cache hit (standard)	0.1x (10%) base	—	—
Cache hit (Fable 5.1)	0.025x (2.5%) base	—	—
Cache hit (Opus 5.5)	0.05x (5%) base	—	—

Batch Processing

50% Discount on All Token Costs

Process large volumes asynchronously. Example: Opus 5.5 batch input = \$2/MTok (vs \$4 standard). Batch + prompt caching discounts **stack** together.

Data Residency & Fast Mode

Feature	Multiplier / Cost	Details
US-only inference (<code>inference_geo: "us"</code>)	1.1x all token costs	Claude 4.6+ only. Global (default) = standard.
Fast mode — Opus 5.5	\$8 in / \$40 out per MTok	2x standard price. Stacks with other multipliers.
Fast mode — Opus 5/4.8	\$10 in / \$50 out per MTok	Research preview, Claude API only.

Tool-Specific Pricing

Tool	Additional Cost
Web Search	\$10 per 1,000 searches + tokens
Web Fetch	No extra cost (tokens only)
Code Execution	Free with web search/fetch; else \$0.05/hr after 1,550 free hrs/mo
Managed Agents runtime	\$0.08 per session-hour (only while <code>running</code>)

Cost Optimization — Example Calculation

Worked Example: 1-Hour Coding Session with Opus 5

Line Item	Calculation	Cost
Input tokens (50K)	$50,000 \times \$5 / 1,000,000$	\$0.25
Output tokens (15K)	$15,000 \times \$25 / 1,000,000$	\$0.375
Session runtime	$1.0 \text{ hr} \times \$0.08$	\$0.08
Total		\$0.705

With prompt caching (40K cached reads): total drops to **\$0.525** — a 25% saving.

PART 05

Messages API — Fundamentals

Request Structure — Example

```
{ "model": "claude-opus-5-5", // REQUIRED "max_tokens": 1024, // REQUIRED "messages": [ // JSON  
  REQUIRED { "role": "user", "content": "What is the capital of France?" } ], "system": "You are a  
  geography expert.", // Optional "temperature": 0.7, // Optional (0-1) "tools": [...], //  
  Optional "tool_choice": {"type": "auto"} // Optional }
```

Python SDK Example

```
import anthropic client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY from env message PYTHON  
client.messages.create( model="claude-opus-5-5", max_tokens=1024, system="You are a helpful  
coding assistant.", messages=[ {"role": "user", "content": "Explain Python decorators."} ], )  
print(message.content[0].text)
```

Stop Reasons

Stop Reason	Meaning	What To Do
end_turn	Claude finished naturally	Display response to user
max_tokens	Hit the max_tokens limit	Consider increasing max_tokens
stop_sequence	Hit a custom stop sequence	Process the partial response
tool_use	Claude wants to call tool(s)	Execute tool, return tool_result

Content Block Types

Type	In Request?	In Response?	Purpose
text	Yes	Yes	Plain text content
image	Yes	No	Image input (base64, URL, file_id)
tool_use	No	Yes	Claude's tool call request
tool_result	Yes	No	Your tool execution results

Type	In Request?	In Response?	Purpose
thinking	No	Yes	Claude's reasoning steps
document	Yes	No	PDF or document content

Authentication Best Practices

- Set **expiration** when creating API keys
- Keep keys **out of source control**
- Never put keys in **client-side code** or prompts
- Consider **Workload Identity Federation** as alternative to static keys
- Headers: `x-api-key` + `anthropic-version: 2023-06-01`

PART 06

Tool Use & Function Calling

How Tool Use Works

The Tool Use Loop

- 1. Define tools (name, description, input_schema)
- 2. Send request → Claude returns `tool_use` block
- 3. Your code executes the tool
- 4. Send `tool_result` back to Claude
- 5. Claude formulates final response (or calls more tools)

Defining a Tool — Example

```
{ "name": "get_weather", "description": "Get the current weather for a location.",  
  "input_schema": { "type": "object", "properties": { "location": { "type": "string",  
    "description": "City and state, e.g. San Francisco, CA" } }, "required": ["location"] } }
```

JSON — TOOL SCHEMA

Complete Tool Use — Python Example

```
# Step 1: Send request with tool definition response = client.messages.create( model="claude-  
opus-5-5", max_tokens=1024, tools=[weather_tool], tool_choice={"type": "auto"}, messages=  
[{"role": "user", "content": "What's the weather in SF?"}], ) # Step 2: Claude returns tool_use  
block tool_use = next(b for b in response.content if b.type == "tool_use") # tool_use.name =  
"get_weather" # tool_use.input = {"location": "San Francisco, CA"} # Step 3: Execute tool & send  
result back weather_data = "15°C, partly cloudy" # your lookup followup =  
client.messages.create( model="claude-opus-5-5", max_tokens=1024, tools=[weather_tool],  
messages=[ {"role": "user", "content": "What's the weather in SF?"}, {"role": "assistant",  
  "content": response.content}, {"role": "user", "content": [ {"type": "tool_result",  
    "tool_use_id": tool_use.id, "content": weather_data} ]}, ], )
```

PYTHON

Tool Choice Modes

Mode	Behavior	Use When
auto	Claude decides (default)	Most use cases
any	Must call <i>some</i> tool	Force tool use on every turn

Mode	Behavior	Use When
<code>tool</code>	Must call a <i>specific</i> named tool	Forcing a particular action
<code>none</code>	No tool calls allowed	Text-only responses

Client-Side vs. Server-Side Tools

Client-Side (You Execute)

- Your own custom tools
- Bash — shell commands
- Text Editor — file manipulation
- Computer Use — desktop control
- Browser Use — web interaction
- Memory — cross-conversation storage

Server-Side (Anthropic Executes)

- Web Search — \$10/1K searches
- Web Fetch — no extra cost
- Code Execution — sandboxed Python/bash
- Advisor — consult smarter model
- Tool Search — dynamic discovery
- MCP Connector — remote MCP servers

Strict Mode

Add `"strict": true` to tool definitions to guarantee Claude's arguments always match your schema exactly — critical for production pipelines.

PART 07

Vision & Multimodal Capabilities

Supported Image Formats

JPEG, PNG, GIF, WebP. Animations unsupported (only first frame used). Claude **cannot** generate, edit, or create images — it is understanding only.

Image Limits

Limit	Value
Max dimensions	8,000 × 8,000 px
Max file size (API)	10 MB
Max file size (Bedrock/GCP)	5 MB
Max per API request (1M context)	600 images
Max per API request (200K context)	100 images
Max per message on claude.ai	20 images

Visual Token Calculation

Formula

Visual tokens = $\lceil \text{width} / 28 \rceil \times \lceil \text{height} / 28 \rceil$

Each 28×28 pixel patch = 1 visual token. A 1000×1000 image = ~1,296 tokens.

Sending Images — Python Example

```
message = client.messages.create( model="claude-opus-5-5", max_tokens=1024, messages=[{"role": "user", "content": [ {"type": "image", "source": { "type": "url", "url": "https://example.com/photo.jpg" } }, {"type": "text", "text": "Describe this image."} ]}], )
```

PYTHON — URL SOURCE

Three image source types: `base64` , `url` , `file` (via Files API `file_id`).

Best Practice

Place images **before** text in messages for best results. Use the Files API for repeated images to avoid re-sending base64 data every turn.

PART 08

Thinking & Reasoning

How Thinking Works

When thinking is active, Claude works through the problem step-by-step before answering. The reasoning arrives in `thinking` content blocks ahead of the `text` response.

Two Thinking Modes

Mode	Config	Models	Behavior
Adaptive	<code>{"type": "adaptive"}</code>	Claude 4.7+ (always on for 5.x)	Claude decides when and how deeply to think. Steered by <code>effort</code> .
Extended	<code>{"type": "enabled", "budget_tokens": N}</code>	Haiku 4.5, older models	Manual mode with explicit budget. Deprecated on 4.6 models.

Configuring Thinking — Example

```
response = client.messages.create( model="claude-opus-5-5", max_tokens=16000, thinking={"type": "adaptive", "display": "summarized"}, messages=[{"role": "user", "content": "What is the GCD of 1071 and 462?" }], ) for block in response.content: if block.type == "thinking": print(f"Thinking: {block.thinking}") elif block.type == "text": print(f"Answer: {block.text}")
```

Key Facts

- Thinking tokens are billed as **output tokens**
- They count toward `max_tokens`
- `display: "summarized"` → visible reasoning text; `"omitted"` → empty thinking field
- Thinking blocks carry a `signature` field — pass back unchanged in multi-turn conversations
- Best for: math, coding, analysis, and agentic tasks

PART 09

Prompt Engineering — Best Practices

The Five Core Principles

1. Be Clear and Direct

Example: Creating an Analytics Dashboard

LESS EFFECTIVE

Create an analytics dashboard

MORE EFFECTIVE

Create an analytics dashboard. Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation.

Golden Rule

Show your prompt to a colleague with minimal context. If they'd be confused, Claude will be too.

2. Add Context and Motivation

Example: Formatting Constraint

LESS EFFECTIVE

NEVER use ellipses

MORE EFFECTIVE

Your response will be read aloud by a text-to-speech engine, so never use ellipses since the TTS engine won't know how to pronounce them.

Claude generalizes from the WHY — it will also avoid other TTS-unfriendly characters.

3. Use Examples (Few-Shot Prompting)

Include **3–5 examples** for best results. Make them:

- **Relevant** — mirror your actual use case

- **Diverse** — cover edge cases
- **Structured** — wrap in `<example>` tags

```
<examples> <example> <input>What is 2+2?</input> <output>4</output> </example> <example> <input>What is the capital of France?</input> <output>Paris</output> </example> </examples> Now answer: <input>{{USER_QUESTION}}</input>
```

PROMPT TEMPLATE

4. Structure with XML Tags

XML tags help Claude parse complex prompts unambiguously:

```
<instructions> Summarize the document in 3 bullet points. Focus on key findings only. </instructions> <document> {{DOCUMENT_TEXT}} </document> <output_format> Return a JSON object with a "summary" array of 3 strings. </output_format>
```

STRUCTURED PROMPT

5. Give Claude a Role

```
// Set via the "system" parameter "You are a senior Python developer with 15 years of experience. You write clean, well-tested code and explain your decisions."
```

SYSTEM PROMPT

Advanced Techniques

Prompt Chaining

Break complex tasks into sequential sub-tasks.
Output of step N becomes context for step N+1.

Structured Outputs

Two approaches: JSON output mode, or `strict: true` on tool definitions for validated schemas.

Long-Context Prompting

Place documents before queries. Use XML tags with `index` attributes for multiple docs.

Effort Parameter

Control thinking depth: `low`, `medium`, `high`. Not supported on Haiku 4.5.

Mnemonic: CERRS

Remember the 5 Principles

Clear and direct instructions
Examples (3-5, diverse, in XML tags)
Role in system prompt

Reasoning context (explain WHY)

Structure with XML tags

PART 10

Safety, Ethics & Data Policies

Anthropic's Three Core Positions

1. Rapid Progress is Likely

Scaling laws show predictable capability improvement. Training costs remain far below major scientific projects.

2. Alignment is Unsolved

"No one knows how to train very powerful AI systems to be robustly helpful, honest, and harmless."

3. Empirical Research is Essential

Problems may only emerge at scale. Safety work must engage with frontier models.

Safety Research Portfolio

Area	Description
Mechanistic Interpretability	Reverse-engineering neural networks to audit for unsafe behaviors
Scalable Oversight	Using AI to help supervise AI training (Constitutional AI)
Process-Oriented Learning	Rewarding safe reasoning steps, not just outcomes
Understanding Generalization	Tracing how training data shapes emergent behaviors

Three Preparedness Scenarios

Scenario	Assumption	Strategy
Optimistic	Current techniques largely sufficient	Continue current approaches
Intermediate	Significant effort can prevent catastrophe	Invest heavily in safety research
Pessimistic	Safety may be unsolvable	Evidence-gathering becomes critical

Anthropic **hedges across all three** rather than betting on one.

Data Retention & Privacy

Zero Data Retention (ZDR)

ZDR-Eligible

- Context windows, thinking, vision
- PDF support, citations, search results
- Structured outputs, prompt caching
- Client-side tools (bash, editor, computer use)
- Token counting, web search*, web fetch*

*Except with dynamic filtering enabled

NOT ZDR-Eligible

- Batch processing
- Code execution
- Files API
- Agent Skills
- MCP connector

Key Privacy Principles

- Retained data is **never used for model training** without express permission
- Anthropic designs for the **smallest possible retention footprint**
- **HIPAA-ready** API access available (same eligibility as ZDR)
- On partner platforms (Bedrock, Google Cloud), the **cloud provider** is the data processor

PART 11

Practice Test — 50 MCQs with Answers

Instructions

Each question has one correct answer highlighted in green. Read the explanation to understand *why* each answer is correct. Covers all exam domains.

Section A — Models & Pricing (Q1–Q10)

Q1 MODELS

Which Claude model is recommended as the default starting point for most workloads?

- A) Claude Fable 5.1
- B) Claude Opus 5.5**
- C) Claude Sonnet 5
- D) Claude Haiku 4.5

Answer: B. The docs state: "If you're unsure which model to use, start with Claude Opus 5.5 for most workloads."

Q2 MODELS

What is the context window size for Claude Haiku 4.5?

- A) 1M tokens
- B) 500K tokens
- C) 200K tokens**
- D) 100K tokens

Answer: C. Haiku 4.5 has a 200K token context window. All other current models have 1M token windows.

Q3 PRICING

What discount does the Batch API provide on standard token costs?

- A) 25%
- B) 50%**
- C) 75%
- D) 30%

Answer: B. The Batch API allows asynchronous processing with a 50% discount on both input and output tokens.

Q4 PRICING

What is the base output token price for Claude Sonnet 5?

- A) \$5 / MTok
- B) \$10 / MTok**
- C) \$15 / MTok
- D) \$20 / MTok

Answer: B. Sonnet 5 is priced at \$2/MTok input and \$10/MTok output — the best speed-to-cost ratio.

Q5 PRICING

What multiplier does the 5-minute prompt cache write incur over base input price?

- A) 1.1x
- B) 1.25x**
- C) 1.5x
- D) 2x

Answer: B. 5-minute cache write = 1.25x base input. 1-hour cache write = 2x base input.

Q6 MODELS

What is the maximum output token limit for Claude Fable 5.1?

- A) 64K tokens
- B) 128K tokens**
- C) 256K tokens
- D) 1M tokens

Answer: B. Fable 5.1, Opus 5.5, and Sonnet 5 all support up to 128K output tokens. Haiku supports 64K.

Q7 PRICING

What is the cache hit cost for Claude Fable 5.1 relative to base input?

- A) 10% (0.1x)
- B) 5% (0.05x)
- C) 2.5% (0.025x)**
- D) 1% (0.01x)

Answer: C. Fable 5.1 cache hits = 2.5% of base (\$0.25/MTok). Standard = 10%. Opus 5.5 = 5%.

Q8 PRICING

Which pricing multiplier applies when using US-only data residency?

- A) 1.0x (no change)
- B) 1.05x
- C) 1.1x**
- D) 1.25x

Answer: C. `inference_geo: "us"` incurs a 1.1x multiplier on all token pricing categories.

Q9 PLATFORMS

How many platforms currently offer Claude models?

A) Three

B) Four

C) Five

D) Six

Answer: C. Claude API, Amazon Bedrock, Claude Platform on AWS, Google Cloud/Vertex AI, Microsoft Foundry.

Q10 MODELS

What is the default effort level for Claude Opus 5.5?

A) low

B) medium

C) high

D) Not supported

Answer: B. Opus 5.5 defaults to "medium." Fable 5.1 and Sonnet 5 default to "high." Haiku doesn't support it.

Section B — API & Tools (Q11–Q20)

Q11 API

What `stop_reason` indicates Claude wants to call a tool?

A) `end_turn`

B) `tool_use`

C) `stop_sequence`

D) `max_tokens`

Answer: B. The response has `stop_reason: "tool_use"` and contains `tool_use` content blocks.

Q12 TOOLS

Which `tool_choice` type forces Claude to call a specific named tool?

- A) auto
- B) any
- C) tool**
- D) required

Answer: C. `tool` forces a specific tool; `any` forces some tool; `auto` lets Claude decide.

Q13 TOOLS

Server-side tools run on which infrastructure?

- A) The user's application server
- B) Anthropic's infrastructure**
- C) AWS Lambda
- D) The client browser

Answer: B. Server tools (web search, web fetch, code execution) run on Anthropic's infrastructure.

Q14 PRICING

What is the per-search cost for the Web Search tool?

- A) \$1 per 1,000 searches
- B) \$5 per 1,000 searches
- C) \$10 per 1,000 searches**
- D) Free

Answer: C. Web search = \$10 per 1,000 searches, plus standard token costs for search content.

Q15 PRICING

What additional cost does the Web Fetch tool incur beyond tokens?

A) \$5 per 1,000 fetches

B) \$0.01 per fetch

C) No additional cost

D) \$10 per 1,000 fetches

Answer: C. Web fetch has no additional charges — you only pay token costs for fetched content.

Q16 API

Which parameter sets the system prompt in the Messages API?

A) context

B) instructions

C) system

D) prompt

Answer: C. The `system` parameter sets the system prompt for behavior and context.

Q17 PRICING

What is the session runtime charge for Claude Managed Agents?

A) \$0.04 per session-hour

B) \$0.08 per session-hour

C) \$0.15 per session-hour

D) Free

Answer: B. \$0.08/session-hour, metered to the millisecond only while status is `running` .

Q18 API

Which content block type must you send back after executing a tool?

- A) tool_output
- B) tool_response
- C) tool_result**
- D) function_result

Answer: C. Send a `tool_result` block with the `tool_use_id` and result content.

Q19 TOOLS

What is the MCP connector used for?

- A) Managing Claude's memory
- B) Connecting to remote MCP servers from the API**
- C) Monitoring Claude's performance
- D) Compressing prompts

Answer: B. The MCP connector connects to remote Model Context Protocol servers directly from the Messages API.

Q20 TOOLS

Which is NOT an Anthropic-schema client tool?

- A) Bash
- B) Text Editor
- C) Web Search**
- D) Computer Use

Answer: C. Web Search is a *server-side* tool. The others are client-side tools.

Section C — Prompt Engineering (Q21–Q30)

Q21 PROMPTING

What is Anthropic's "golden rule" for prompt clarity?

- A) Use the longest prompt possible
- B) Show it to a colleague with minimal context**
- C) Always use XML tags
- D) Include at least 10 examples

Answer: B. If a colleague with minimal context would be confused, Claude will be too.

Q22 PROMPTING

How many examples does Anthropic recommend for few-shot prompting?

- A) 1–2
- B) 3–5**
- C) 7–10
- D) 15+

Answer: B. 3–5 examples, made relevant, diverse, and wrapped in `<example>` tags.

Q23 PROMPTING

What format does Anthropic recommend for structuring complex prompts?

- A) JSON
- B) YAML
- C) XML tags**
- D) Markdown headers

Answer: C. XML tags help Claude parse prompts unambiguously: `<instructions>` , `<context>` , `<input>` .

Q24 THINKING

What is "adaptive thinking" in Claude 4.7+?

- A) A fixed thinking template
- B) Claude dynamically decides when and how much to think**
- C) User manually sets budget_tokens
- D) Thinking is always disabled

Answer: B. Adaptive thinking = Claude dynamically decides. It's the only mode on 4.7+.

Q25 PROMPTING

Where should role information be placed?

- A) In the first user message
- B) In the system prompt**
- C) As a tool definition
- D) In stop_sequences

Answer: B. Set roles in the system prompt to focus Claude's behavior and tone.

Q26 PROMPTING

What is prompt chaining?

- A) Multiple simultaneous API calls
- B) Breaking complex tasks into sequential sub-tasks**
- C) Linking multiple models together
- D) Caching prompts across sessions

Answer: B. Each step focuses on one aspect, passing its output as context to the next.

Q27 PROMPTING

Why does adding context/motivation improve prompts?

- A) It makes prompts longer
- B) Claude generalizes from the explanation**
- C) It forces XML tag usage
- D) It reduces token count

Answer: B. Claude is smart enough to generalize from the WHY behind a constraint.

Q28 FEATURES

What does the `effort` parameter control?

- A) Output format
- B) How many tokens Claude uses when responding**
- C) The model's temperature
- D) API rate limits

Answer: B. Trades off between response thoroughness and token efficiency.

Q29 PROMPTING

What tags should wrap few-shot examples in prompts?

- A) `<sample>`
- B) `<demo>`
- C) `<example>`**
- D) `<test>`

Answer: C. Wrap in `<example>` tags (multiple in `<examples>`) to distinguish from instructions.

Q30 TOOLS

What does `strict: true` guarantee in a tool definition?

- A) The tool always executes
- B) Claude's arguments match your schema exactly**
- C) The tool runs server-side
- D) Error messages are suppressed

Answer: B. Guarantees Claude's tool call arguments always conform to your JSON schema.

Section D — Features (Q31–Q40)

Q31 FEATURES

Which feature grounds responses in source documents with exact references?

- A) Structured outputs
- B) Citations**
- C) Search results
- D) Prompt caching

Answer: B. Citations provide exact references to sentences and passages for verifiable outputs.

Q32 VISION

What image formats does Claude's vision support?

- A) JPEG and PNG only
- B) JPEG, PNG, GIF, and WebP**
- C) All image formats
- D) JPEG, PNG, and SVG

Answer: B. JPEG, PNG, GIF, WebP. Animations unsupported (only first frame).

Q33 VISION

What is the maximum image dimension Claude accepts?

- A) 4096 × 4096 px
- B) 8000 × 8000 px**
- C) 2576 × 2576 px
- D) 1920 × 1080 px

Answer: B. Maximum dimensions per image are 8000 × 8000 px.

Q34 VISION

How are visual tokens calculated for images?

- A) Total pixels / 1000
- B) $\lceil \text{width}/28 \rceil \times \lceil \text{height}/28 \rceil$**
- C) Width × Height
- D) File size in KB

Answer: B. Each 28×28 pixel patch = 1 visual token.

Q35 FEATURES

What does "compaction at a token threshold" do?

- A) Compresses images
- B) Summarizes earlier conversation when tokens reach trigger**
- C) Reduces prompt cache size
- D) Removes stop sequences

Answer: B. Server-side context summarization for long-running conversations.

Q36 FEATURES

Which feature processes large request volumes asynchronously?

- A) Streaming
- B) Prompt caching

C) Batch processing

- D) Fast mode

Answer: C. Batch processing = async processing at 50% token discount.

Q37 FEATURES

How many approaches does structured outputs offer?

- A) One

B) Two

- C) Three

- D) Four

Answer: B. JSON outputs for structured data, and strict tool use for validated inputs.

Q38 VISION

Which source types can provide images in the API?

- A) Base64 only

- B) URL only

C) Base64, URL, and file_id

- D) Base64 and URL only

Answer: C. Three types: base64-encoded, URL reference, and file_id from the Files API.

Q39 FEATURES

What does the Token Counting API help you do?

- A) Count words in responses
- B) Determine token count before sending**
- C) Track billing automatically
- D) Measure response latency

Answer: B. Lets you determine token count before sending — useful for prompt optimization.

Q40 VISION

Can Claude generate, edit, or create images?

- A) Yes, all models can
- B) Only Fable 5.1 can
- C) No, understanding only**
- D) Yes, with the Vision tool

Answer: C. Claude is image understanding only — it cannot generate, edit, or create images.

Section E — Safety & Data (Q41–Q50)

Q41 SAFETY

What is Anthropic's position on using customer data for model training?

- A) All data is used
- B) Used unless opted out
- C) Never used without express permission**
- D) Only anonymized data

Answer: C. Retained data is never used for model training without express permission.

Q42 DATA

What does ZDR stand for?

- A) Zero Downtime Recovery
- B) Zero Data Retention**
- C) Zero Delay Response
- D) Zone Data Replication

Answer: B. Zero Data Retention — no storage of customer prompts or responses for eligible features.

Q43 DATA

Which feature is NOT ZDR-eligible?

- A) Context windows
- B) Vision
- C) Batch processing**
- D) PDF support

Answer: C. Batch processing is NOT ZDR-eligible. The others are.

Q44 SAFETY

What are Anthropic's three preparedness scenarios?

- A) Small, medium, large
- B) Optimistic, intermediate, pessimistic**
- C) Internal, external, hybrid
- D) Current, near-term, long-term

Answer: B. Optimistic (techniques suffice), intermediate (effort prevents catastrophe), pessimistic (may be unsolvable).

Q45 SAFETY

Which research area involves reverse-engineering neural networks?

- A) Scalable oversight
- B) Process-oriented learning
- C) Mechanistic interpretability**
- D) Understanding generalization

Answer: C. Mechanistic interpretability: reverse-engineering neural networks to audit for unsafe behaviors.

Q46 DATA

On Amazon Bedrock, who is the data processor?

- A) Anthropic
- B) The cloud provider (AWS)**
- C) The customer
- D) A third-party auditor

Answer: B. On Bedrock and Google Cloud, the cloud provider is the data processor. On Claude API, Anthropic is.

Q47 SAFETY

What is Constitutional AI?

- A) A legal compliance framework
- B) A technique using AI to supervise AI training**
- C) A government regulation
- D) A model architecture

Answer: B. Part of scalable oversight — using AI principles to help supervise AI training.

Q48 SAFETY

What does Anthropic say about AI alignment?

- A) Fully solved
- B) Mostly solved with minor gaps
- C) No one knows how to achieve robust alignment**
- D) Impossible to solve

Answer: C. "No one knows how to train very powerful AI systems to be robustly helpful, honest, and harmless."

Q49 DATA

Does Anthropic offer HIPAA-ready API access?

- A) No
- B) Yes, with same eligibility as ZDR**
- C) Yes, for all features
- D) Only on Bedrock

Answer: B. Yes, with the same feature eligibility structure as ZDR.

Q50 SAFETY

Why does safety research require frontier models?

- A) Smaller models are too expensive
- B) Problems may only emerge at scale**
- C) Legal requirements mandate it
- D) Smaller models lack safety features

Answer: B. Large models behave qualitatively differently — safety problems may only emerge at scale.

All content sourced from official Anthropic documentation (platform.claude.com/docs).

Last updated September 2026. Compiled for educational purposes.